ASSIGNMENT 2

Subject Name: Exploratory Data Analysis
Subject Code: BAI515E

Submitted By:

Pramod Biradar
1AY25AI407

Submitted To:

Ms. Surbhi

Table of Contents

Sl. No.	Topic	Page No.
1	Introduction	4
2	Data Loading and Inspection	4
3	Pivot Table by Decade and Gender	5
4	Data Cleaning with Sigma-Clipping	7
5	Datetime Index and Weekday Analysis	9
6	Births by Day of the Year	10
7	Pivot Table versus groupby	12
8	Bonus: Lowest Birth Dates and Annotated Plot	14
9	Conclusion	15
10	References	15

LIST OF FIGURES

SL.NO	TITLE	PAGE.NO
1	Total births per year by gender	7
2	Mean daily births by day of week for each decade	10
3	Average daily births by calendar date	12
4	Average daily births with three holiday dips labelled	15

LIST OF TABLES

SL.NO	TITLE	PAGE.NO
1	Total births by decade and gender	5
2	Male-to-female ratio and percentage by which male births exceed female births	6
3	Mean daily births by day of week and decade	9
4	Decade-by-gender totals computed with groupby and unstack	13
5	Timing comparison of pivot_table and groupby + unstack	13
6	Five dates with the lowest mean births and matching holidays	14

1. Introduction

Birth records make a handy dataset for exploratory work: every row has a calendar position (year, month and day), a category (gender) and one numeric count. This assignment studies the CDC birth data for the United States using Python, Pandas and NumPy. The guiding questions are straightforward but telling: do boys outnumber girls consistently, how do daily totals vary across weekdays and across the year, and what happens on public holidays? On the way, the report exercises pivot tables, outlier removal by sigma-clipping, datetime indexing, and a head-to-head comparison of pivot_table with groupby.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100
URL = 'https://raw.githubusercontent.com/jakevdp/data-CDCbirths/master/births.csv'
```

2. Task 1: Data Loading and Inspection

The analysis starts by reading the CSV straight from its repository URL and examining it before any transformation. Since a later task needs the data in its original state, a pristine copy named bdf_raw is saved immediately after loading, while every cleaning step is applied to bdf alone. The inspection displays the first five rows, the table's dimensions, each column's data type and the number of blank cells per column.

```
bdf = pd.read_csv(URL)
bdf_raw = bdf.copy()
print(bdf.head())
print("Shape:", bdf.shape)
print(bdf.dtypes)
print("Missing values per column:")
print(bdf.isna().sum())
```

```
   year  month  day gender  births
0  1969      1  1.0      F    4046
1  1969      1  1.0      M    4440
2  1969      1  2.0      F    4454
3  1969      1  2.0      M    4548
4  1969      1  3.0      F    4548
Shape: (15547, 5)
year        int64
month       int64
day       float64
gender     object
births      int64
dtype: object
Missing values per column:
year        0
month       0
day       480
gender      0
births      0
dtype: int64
```

Output 1: First rows, shape, dtypes and missing-value counts

Why day is stored as a float

Output 1 reports 15,547 rows, and of the four columns only day is float64; year, month and births are integers. This comes down to a NumPy restriction: integer arrays cannot hold a missing entry, whereas NaN exists only as a floating-point value. The day column contains 480 blanks, since from 1989 the CDC released only the birth month. Pandas therefore promotes the entire column to float64, so a date like the 5th shows up as 5.0.

3. Task 2: Pivot Table by Decade and Gender

To bucket years into decades, each year is rounded down to a multiple of ten using 10 * (year // 10), so 1987 becomes 1980. A pivot table then lists the decades as rows and the genders as columns, summing births in every cell. Because this task concerns totals over the whole file, it works on the uncleaned copy bdf_raw, which also explains why the yearly plot below extends to 2008.

(a) Do male births outnumber female births in every decade?

```
bdf_raw['decade'] = (bdf_raw['year'] // 10) * 10
by_decade = bdf_raw.pivot_table(values='births', index='decade',
                                columns='gender', aggfunc='sum')
print(by_decade)

print("Male > Female in every decade:", (by_decade['M'] >
by_decade['F']).all())
```

Table 1: Total births by decade and gender

```
gender         F         M
decade
1960     1753634   1846572
1970    16263075  17121550
1980    18310351  19243452
1990    19479454  20420553
2000    18229309  19106428
Male > Female in every decade: True
```

Setting the M column against the F column row by row and applying .all() reduces the five comparisons to a single result, and that result is True: boys outnumber girls in the 1960s, 1970s, 1980s, 1990s and 2000s. A warning applies when reading the table. The file covers 1969 to 2008, so the 1960 row consists of just one year, the 2000 row of nine years and every other row of ten. Row totals therefore cannot be compared with each other, but the male and female values within a single row span identical years and can be compared safely.

(b) Male-to-female ratio and percentage excess

```
ratio_tbl = pd.DataFrame({
    'M/F ratio': by_decade['M'] / by_decade['F'],
    '% male excess': (by_decade['M'] / by_decade['F'] - 1) * 100
})
print(ratio_tbl.round(4))
```

Table 2: Male-to-female ratio and percentage by which male births exceed female births

```
         M/F ratio  % male excess
decade
1960        1.0530         5.2997
1970        1.0528         5.2787
1980        1.0510         5.0960
1990        1.0483         4.8312
2000        1.0481         4.8116
```

The ratio is the male total divided by the female total; subtracting one and multiplying by 100 turns it into a percentage excess. All five ratios fall between 1.048 and 1.053, meaning boys exceed girls by about 4.8% to 5.3%. The ratio drifts down slightly from the 1960s to the 2000s, yet it stays strikingly constant. It reflects the familiar human sex ratio at birth of roughly 105 boys for every 100 girls, and agrees with Figure 1, where for example 1990 shows about 2.13 million boys against 2.03 million girls.

(c) Total births per year by gender

```
yr_tot = bdf_raw.pivot_table(values='births', index='year', columns='gender',
                             aggfunc='sum')
ax = yr_tot.plot(figsize=(8, 4.5))
ax.set_xlabel('Year')
ax.set_ylabel('Total births')
ax.set_title('Total births per year by gender')
ax.legend(title='Gender')
plt.savefig('fig_births_per_year.png', dpi=150, bbox_inches='tight')
plt.show()

print("Average yearly gap (M - F):", (yr_tot['M'] - yr_tot['F']).mean().round(0))
print("Average gap as % of female births:", ((yr_tot['M'] / yr_tot['F'] - 1) *
100).mean().round(2))
```

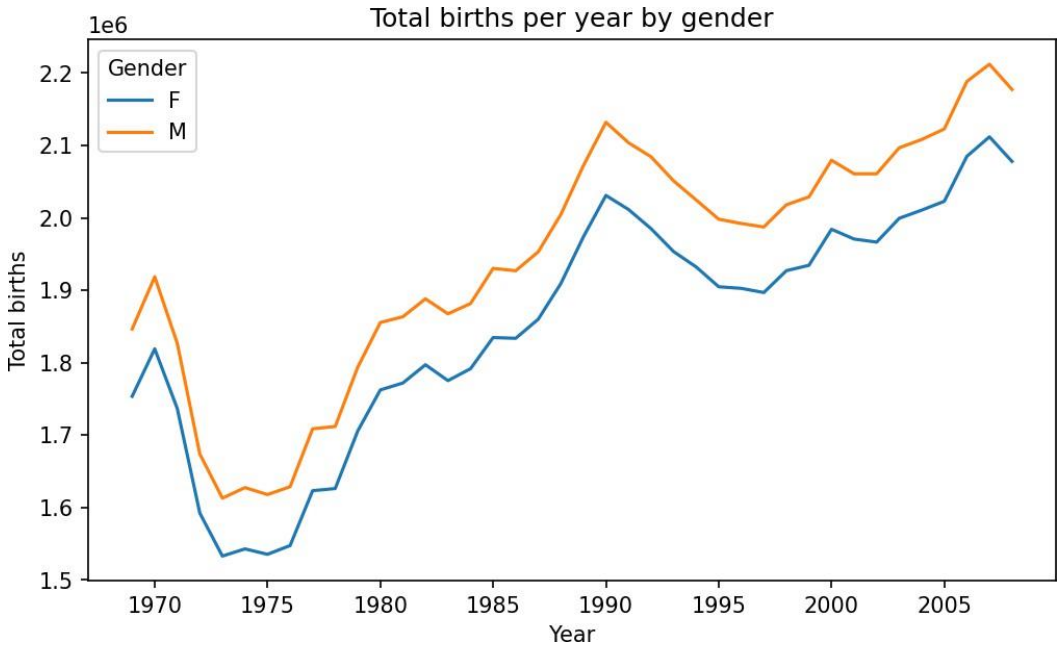


Figure 1: Total births per year by gender

The first thing to notice in Figure 1 is that the two curves run almost in parallel. Whatever moves the national birth count (economic conditions, family planning, the size of the parent generation) affects boys and girls in the same way. Births dropped sharply after 1970 and reached their lowest point in 1973 to 1975, at roughly 1.53 million girls and 1.61 million boys. They then rebounded to a peak around 1990, eased off until about 1997 and rose again through 2007. Throughout, the male curve stays above the female one, and the annual gap of about 80,000 to 100,000 births remains close to 5% of the female count, so the proportion holds steady even while the absolute figures swing widely.

4. Task 3: Data Cleaning with Sigma-Clipping

Not all rows describe a genuine day. Some hold impossible dates such as June 31, and others lack a usable day altogether (for example the placeholder day 99). Their births values are abnormal: an impossible date gathers only a handful of stray births, while a row with no day, as seen from 1989 onward, acts as a whole-month total far above any single day. As both types sit far outside the normal daily range, a sigma-clipping rule on the births column can flag them without checking dates one at a time.

(a) Quartiles, robust mean and robust standard deviation

```
q1, q2, q3 = np.quantile(bdf['births'], [0.25, 0.50, 0.75])
center = q2
spread = 0.74 * (q3 - q1)
print(f"Q1 = {q1}, median = {q2}, Q3 = {q3}")
print(f"mu = {center}, sigma = {spread:.2f}")
print(f"Accepted range: {center - 5*spread:.1f} to {center + 5*spread:.1f}")
```

```
Q1 = 4358.0, median = 4814.0, Q3 = 5289.5
mu = 4814.0, sigma = 689.31
Accepted range: 1367.5 to 8260.5
```

Output 2: Quartiles, mu and sigma

The centre of the data is taken as the median (4,814), and the spread is estimated from the interquartile range instead of the ordinary standard deviation. The factor 0.74 follows from a short derivation. For a normal distribution with standard deviation s, the 25th and 75th percentiles lie about 0.6745 s either side of the mean, so the IQR is 1.349 s. Solving gives s = IQR / 1.349 = 0.7413 x IQR, which rounds to 0.74. The resulting estimate of 689.31 describes the spread of typical rows, and the accepted band of 1,367.5 to 8,260.5 is wide enough to retain legitimate days while rejecting the extreme ones.

(b) Keeping rows within 5 sigma of mu

```
n_before = len(bdf)
bdf = bdf.query('(births > @center - 5 * @spread) & '
                '(births < @center + 5 * @spread)').copy()
print("Rows before:", n_before)
print("Rows after :", len(bdf))
print("Rows removed:", n_before - len(bdf))
```

```
Rows before: 15547
Rows after : 14610
Rows removed: 937
```

Output 3: Number of rows before and after clipping

DataFrame.query() filters rows using a string expression, and prefixing a name with @ makes it read an ordinary Python variable rather than a column. Only rows lying strictly inside the band are kept. Of the 15,547 original rows, 14,610 remain, so **937 rows** were dropped. That figure covers the 480 rows without a day as well as the impossible-date rows.

(c) Converting day to integers and why a robust estimate is used

```
bdf['day'] = bdf['day'].astype(int)
print(bdf.dtypes)
print("Missing values left:", bdf.isna().sum().sum())
```

```
year       int64
month      int64
day        int32
gender    object
births     int64
dtype: object
Missing values left: 0
```

Output 4: Column dtypes after the conversion

Calling astype(int) on the raw column would fail with an error because NaN has no integer counterpart. After clipping, however, every row with a missing day has been removed, no blank cell remains, and the conversion succeeds. Output 4 shows day as an integer type (int32 in this run; the exact width depends on the platform).

Using the median and IQR rather than the mean and standard deviation is the key design choice. Outliers distort the ordinary mean and standard deviation, which are the very quantities that would define what counts as an outlier: a handful of huge monthly totals would lift the mean, inflate the standard deviation and widen the acceptance band until some bad rows slip through. Order statistics avoid this problem. The median and quartiles depend only on the position of values in the sorted list, so they remain stable as long as fewer than about a quarter of the observations are corrupted. That makes the cut-off dependable.

5. Task 4: Datetime Index and Weekday Analysis

(a) DatetimeIndex and day of week

Once the columns are clean integers, pd.to_datetime can merge year, month and day into one timestamp per row, and these timestamps become the table's index. A DatetimeIndex provides calendar attributes, so the weekday (Monday is 0, Sunday is 6) is taken from it and saved in a new column. The decade column is built again on the cleaned table, since the earlier one exists only on the raw copy.

```
bdf.index = pd.to_datetime(bdf[['year', 'month', 'day']])
bdf['dayofweek'] = bdf.index.dayofweek
bdf['decade'] = (bdf['year'] // 10) * 10
print(bdf.head())
print("Date range:", bdf.index.min().date(), "to", bdf.index.max().date())
```

```
            year  month  day gender  births  dayofweek  decade
1969-01-01  1969      1    1      F    4046          2    1960
1969-01-01  1969      1    1      M    4440          2    1960
1969-01-02  1969      1    2      F    4454          3    1960
1969-01-02  1969      1    2      M    4548          3    1960
1969-01-03  1969      1    3      F    4548          4    1960
Date range: 1969-01-01 to 1988-12-31
```

Output 5: Cleaned data with DatetimeIndex and dayofweek

(b) Mean births by day of week and decade

```
wk_tab = bdf.pivot_table(values='births', index='dayofweek',
                         columns='decade', aggfunc='mean')
print(wk_tab.round(1))

ax = wk_tab.plot(figsize=(8, 4.5), marker='o')
ax.set_xticks(range(7))
ax.set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])
ax.set_xlabel('Day of week')
ax.set_ylabel('Mean daily births')
ax.set_title('Mean daily births by day of week and decade')
ax.legend(title='Decade')
plt.savefig('fig_births_by_weekday.png', dpi=150, bbox_inches='tight')
plt.show()
```

Table 3: Mean daily births by day of week and decade

decade	1960	1970	1980
dayofweek			
0	5063.8	4689.1	5276.9
1	5286.1	4885.3	5503.8
2	5074.6	4750.4	5367.6
3	4978.3	4696.9	5333.5
4	5107.9	4782.1	5393.1
5	4651.1	4207.8	4483.9
6	4342.3	3979.3	4308.1

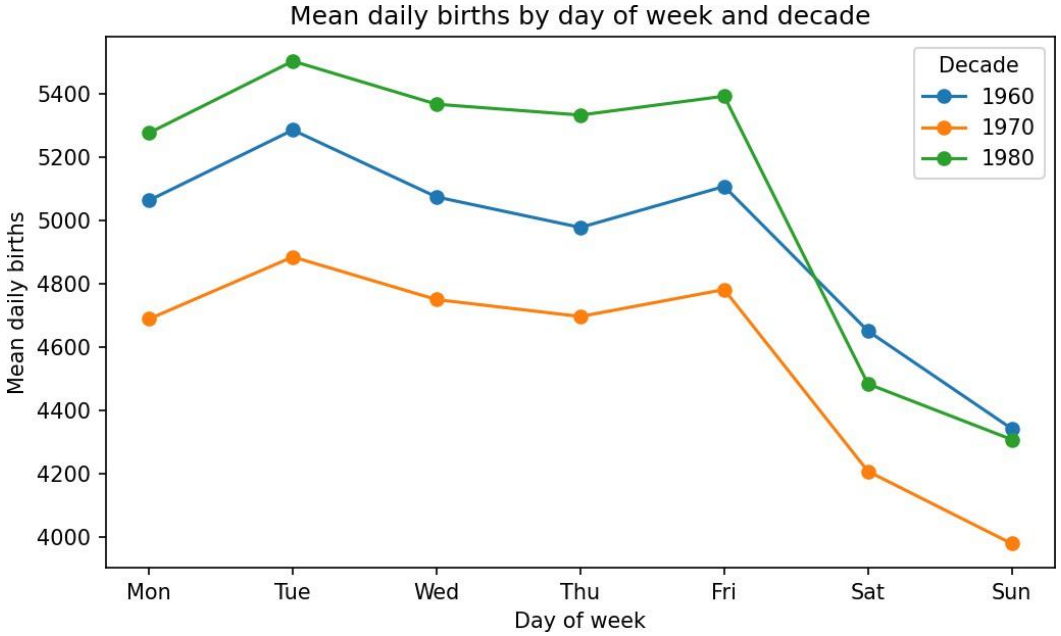


Figure 2: Mean daily births by day of week for each decade

(c) Weekend dip and the missing decades

All three lines in Figure 2 share one shape: a high plateau from Monday to Friday, a peak on Tuesday, and a steep fall on Saturday and Sunday, with Sunday the lowest. In the 1980s, weekday averages run from roughly 5,300 to 5,500 births, whereas Saturday drops to about 4,500 and Sunday to about 4,300. If labour began at random, no day of the week would stand out. The pattern instead suggests medical scheduling: caesarean sections and induced labours are planned ahead and mostly fall on working days, when hospitals are fully staffed, leaving fewer deliveries for weekends.

Figure 2 shows only three decades even though Figure 1 extends to 2008. The cause is the cleaning step. From 1989 the CDC file has no day value, and those rows hold whole-month counts that the 5-sigma filter removed. After clipping, the table spans 1969 to 1988, leaving the decades 1960, 1970 and 1980; the 1960 curve rests on 1969 alone and the 1980 curve on 1980 to 1988.

6. Task 5: Births by Day of the Year

(a) From a (month, day) multi-index to dates

The next question disregards the year and asks what a typical day looks like across the calendar. Pivoting on index=["month", "day"] averages the births for each calendar date over all years and both genders. To obtain a time axis, the two index levels are converted into regular columns, a constant year of 2012 is added, and pd.to_datetime constructs real dates from the three columns in a single vectorised call.

```
daily_avg = bdf.pivot_table(values='births', index=['month', 'day'],
                            aggfunc='mean')
print(daily_avg.head())

dates = daily_avg.index.to_frame(index=False)
dates['year'] = 2012
daily_avg.index = pd.to_datetime(dates[['year', 'month', 'day']])
print(daily_avg.head())
print("Number of calendar dates:", len(daily_avg))
```

```
             births
month day
1     1    4009.225
      2    4247.400
      3    4500.900
      4    4571.350
      5    4603.625
               births
2012-01-01  4009.225
2012-01-02  4247.400
2012-01-03  4500.900
2012-01-04  4571.350
2012-01-05  4603.625
Number of calendar dates: 366
```

Output 6: Mean births by date before and after the date conversion

The placeholder year has to be a leap year. The data contains February 29 records, and the pair (2, 29) cannot be converted in a non-leap year such as 2013, where pd.to_datetime would raise an error; dropping the row to avoid it would quietly discard a real date. The year 2012 includes all 366 possible dates, and the printed count of 366 confirms nothing was lost.

(b) Plot of mean births by date

```
fig, ax = plt.subplots(figsize=(12, 4))
daily_avg.plot(ax=ax, legend=False)
ax.set_xlabel('Date (dummy year 2012)')
ax.set_ylabel('Mean daily births')
ax.set_title('Average daily births by calendar date')
plt.savefig('fig_births_by_date.png', dpi=150, bbox_inches='tight')
plt.show()
```

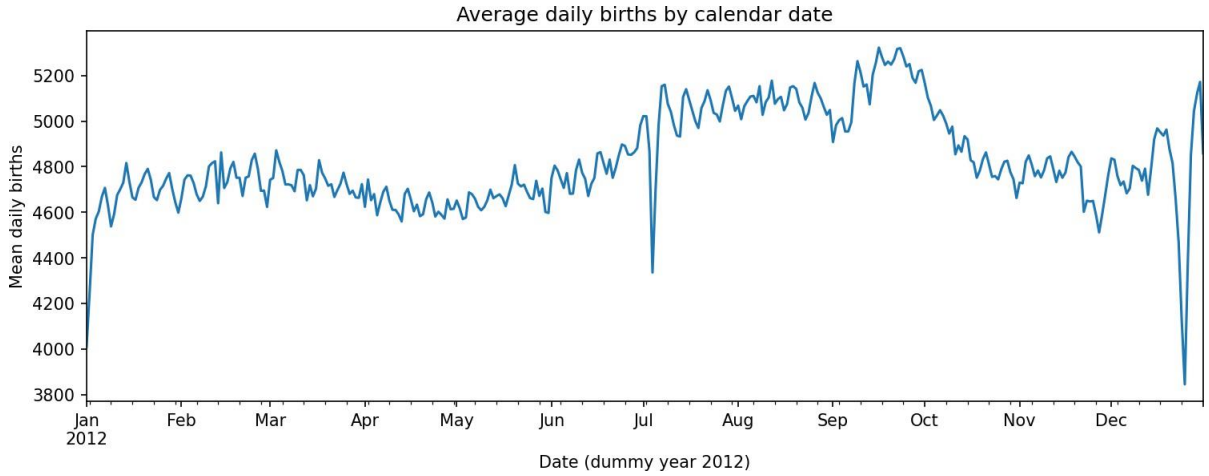


Figure 3: Average daily births by calendar date

(c) Holiday dips

```
print(daily_avg['births'].nsmallest(10).round(1))
```

```
2012-12-25    3844.4
2012-01-01    4009.2
2012-12-24    4126.2
2012-01-02    4247.4
2012-07-04    4335.3
2012-12-26    4383.5
2012-12-23    4466.7
2012-01-03    4500.9
2012-11-27    4511.6
2012-01-09    4537.8
Name: births, dtype: float64
```

Output 7: Ten calendar dates with the lowest mean births

A yearly profile like this combines a slow seasonal movement with a few sharp downward spikes. The slow part is a climb from July to a maximum in mid-September, which is unrelated to any holiday. The spikes are New Year's Day (about 4,000 births compared with a typical winter level near 4,700), Independence Day (about 4,330) and, deepest of all, Christmas Day (about 3,850). The neighbouring days, such as Christmas Eve and January 2, are depressed as well. Floating holidays, such as Thanksgiving on the fourth Thursday of November and Labor Day on the first Monday of September, fall on different dates in different years, so averaging by date smears them into weaker, wider dips.

Biology cannot account for a drop on particular dates, since a baby is no less likely to be born on December 25. The cause is human scheduling: planned caesareans and inductions are seldom booked on holidays when staff are off, and are often shifted to the days before or after. It is the same mechanism behind the weekend dip in Task 4.

7. Task 6: Pivot Table versus groupby

(a) The decade-by-gender table using groupby

```
gb_tab = bdf_raw.groupby(['decade', 'gender'])['births'].sum().unstack()
print(gb_tab)
print("Identical to pivot table:", gb_tab.equals(by_decade))
pd.testing.assert_frame_equal(gb_tab, by_decade, check_names=False)
```

Table 4: Decade-by-gender totals computed with groupby and unstack

```
gender          F         M
decade
1960      1753634   1846572
1970     16263075  17121550
1980     18310351  19243452
1990     19479454  20420553
2000     18229309  19106428
Identical to pivot table: True
```

The same table can be built in two steps. Grouping by decade and gender and summing births gives a Series with a two-level index, and unstack() moves the inner level (gender) into the columns. Both the boolean check equals() and the stricter assert_frame_equal() confirm that this table is identical to the pivot table in Table 1. Both were computed from bdf_raw, so the two methods are compared on the same input.

(b) Timing the two approaches

```
t_pivot = %timeit -o bdf_raw.pivot_table('births', index='decade',
                                         columns='gender', aggfunc='sum')
t_group = %timeit -o bdf_raw.groupby(['decade',
                                      'gender'])['births'].sum().unstack()

timing = pd.DataFrame({
    'Method': ['pivot_table', 'groupby + unstack'],
    'Mean time (ms)': [t_pivot.average * 1000, t_group.average * 1000],
    'Std dev (ms)': [t_pivot.stdev * 1000, t_group.stdev * 1000],
}).round(3)
timing['Relative speed'] = (timing['Mean time (ms)'] / timing['Mean time (ms)'].min()).round(2)
print(timing.to_string(index=False))
```

```
5.9 ms ± 839 µs per loop (mean ± std. dev. of 7 runs, 100 loops e
2.79 ms ± 532 µs per loop (mean ± std. dev. of 7 runs, 100 loops
```

Output 8: %timeit results

Table 5: Timing comparison of pivot_table and groupby + unstack

Method	Mean time (ms)	Std dev (ms)	Relative speed
pivot_table	5.899	0.839	2.11
groupby + unstack	2.795	0.532	1.00

(c) How a pivot table relates to groupby

Under the hood, a pivot table is a groupby followed by a reshape. Both methods split the data by the key columns, aggregate each group and assemble the results; pivot_table simply does the unstacking itself so that one key becomes the column header. With the same keys and the same aggregation (sum), the output cannot differ, which is why Tables 1 and 4 agree.

The difference lies in interface and cost. pivot_table is a compact, spreadsheet-style call with extras such as margins and fill_value, while groupby is the more general tool: it can aggregate without reshaping, apply several functions or transform groups. Those extras give pivot_table additional work. The timing test shows this, with pivot_table averaging 5.9 ms versus 2.8 ms for groupby + unstack, making it about 2.1 times slower here. For a table this small the absolute gap is negligible, but it could matter on much larger data.

8. Bonus: Lowest Birth Dates and Annotated Plot

```
hol_map = {(1, 1): "New Year's Day", (1, 2): "Day after New Year's",
           (7, 4): "Independence Day", (12, 25): "Christmas Day",
           (12, 24): "Christmas Eve", (12, 31): "New Year's Eve",
           (11, 11): "Veterans Day", (2, 29): "Leap day (few samples)"}

bottom5 = daily_avg['births'].nsmallest(5).to_frame('mean births').round(1)
bottom5['holiday'] = [hol_map.get((d.month, d.day), 'no fixed-date holiday')
                      for d in bottom5.index]
print(bottom5)

fig, ax = plt.subplots(figsize=(12, 4.5))
daily_avg.plot(ax=ax, legend=False)
for (m, d), name, offset in [((1, 1), "New Year's Day", (25, -25)),
                             ((7, 4), "Independence Day", (0, -40)),
                             ((12, 25), "Christmas Day", (-60, -30))]:
    ts = pd.Timestamp(2012, m, d)
    ax.annotate(name, xy=(ts, daily_avg.loc[ts, 'births']),
                xytext=offset, textcoords='offset points',
                arrowprops=dict(arrowstyle='->'), ha='center')
ax.set_xlabel('Date (dummy year 2012)')
ax.set_ylabel('Mean daily births')
ax.set_title('Average daily births with holiday dips labelled')
plt.savefig('fig_births_annotated.png', dpi=150, bbox_inches='tight')
plt.show()
```

Table 6: Five dates with the lowest mean births and matching holidays

Rank	Date	Mean births	Matching holiday
1	December 25	3844.4	Christmas Day
2	January 1	4009.2	New Year's Day
3	December 24	4126.2	Christmas Eve
4	January 2	4247.4	Day after New Year's
5	July 4	4335.3	Independence Day

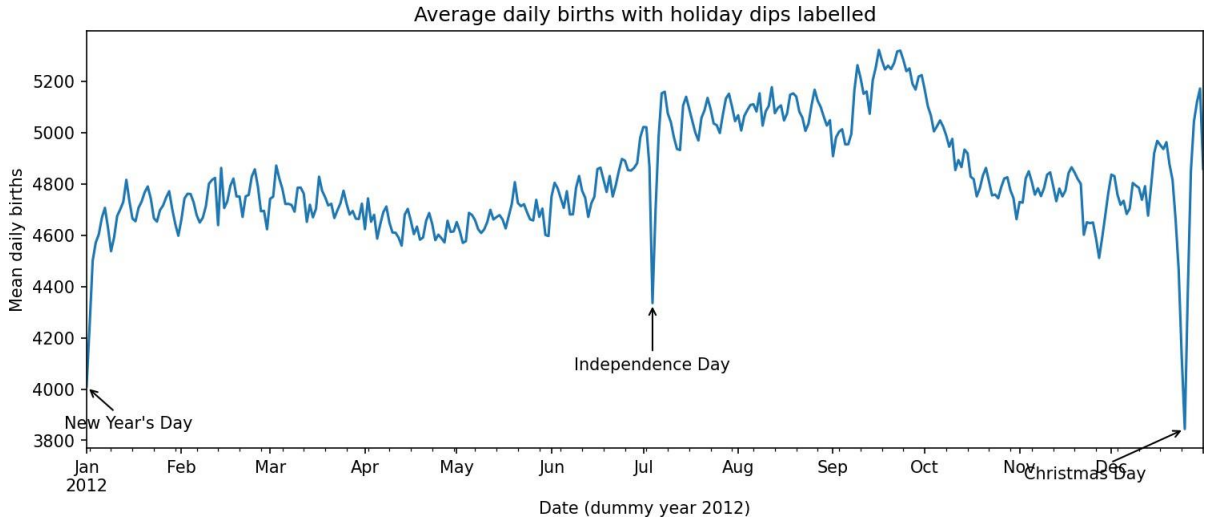


Figure 4: Average daily births with three holiday dips labelled

In order, the five quietest dates are Christmas Day (about 3,844 births), New Year's Day (4,009), Christmas Eve (4,126), January 2 (4,247) and Independence Day (4,335), and each is either a holiday or adjacent to one. Figure 4 uses arrows to mark the three deepest and most clearly isolated: Christmas Day, New Year's Day and Independence Day. The fact that all of the lowest values coincide with holidays supports the explanation from Task 5 that fewer deliveries are scheduled on those days.

9. Conclusion

This study yields three main findings. First, boys are born more often than girls in every decade, by about 5%, and the ratio barely moves over forty years. Second, birth counts are not uniform over time: they are noticeably lower on Saturdays, Sundays and public holidays, a pattern best explained by planned medical procedures rather than biology. Third, on the technical side, a median- and IQR-based sigma-clip removed the corrupted rows reliably, which made an integer day column and a proper datetime index possible; and the pivot_table and groupby + unstack routes produced the same table, with groupby faster in the timing test and pivot_table more convenient to write.

10. References

1. VanderPlas, J. CDC Births Dataset.

2. Pandas Development Team. Pandas Documentation: Pivot Tables.

3. Matplotlib Development Team. Matplotlib Documentation.